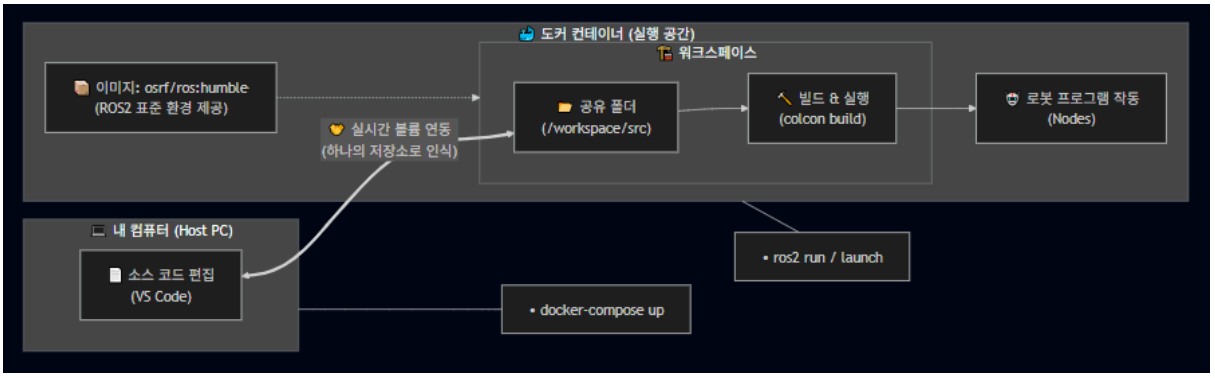
Docker 정리본

🔍 *도커 핵심 개념 정리*

구분	설명
이미지 (Image)	컨테이너 실행에 필요한 모든 파일과 설정이 담긴 **설계도** (Read-only)
컨테이너 (Container)	이미지를 실행한 **실제 상태**. 독립된 프로세스로 동작함
볼륨 (Volume)	컨테이너 삭제 시 사라지는 데이터를 영구 저장하는 공간

예시 이미지



`docker-compose.yml` 파일에서 내 컴퓨터 폴더와 도커 내 폴더를 연결하면, 운영체제가 두 지점을 하나의 저장소처럼 인식한다. 이를 통해 별도의 복사 없이 코드 수정사항이 실시간으로 공유됨.

`osrf/ros:humble-desktop` 이미지가 제공하는 표준 환경에서 별도의 복사 없이 즉시 빌드 및 실행을 수행하는 구조.

🛠️ *도커 기본 명령어*

명령어	설명	주요 옵션 상세
`docker ps`	현재 **실행 중인** 컨테이너 목록 확인	`docker ps -a` : 중지된 컨테이너 포함 전체 확인

`docker run`	이미지를 내려받고 컨테이너를 생성하여 실행	하단 docker 옵션 참고
`docker logs`	컨테이너 내부에서 출력되는 로그 확인	`docker logs [이름/ID]` (실시간 확인은 `-f`)
`docker stop`	실행 중인 컨테이너 정지	`docker stop [이름/ID]`
`docker rm`	정지된 컨테이너 삭제	`docker rm [이름/ID]`

🛠️ *도커 핵심 명령어 요약*

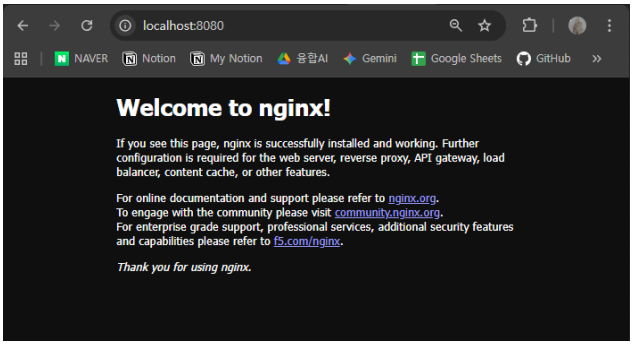
1. 웹 서버 실행 (포트 연결)

```
docker run -d -p 8080:80 --name web nginx
```

- **해석:** 내 컴퓨터 **8080포트**와 컨테이너 **80포트**를 연결하여 Nginx 서버를 백그라운드로 실행함.
- 포트포워딩(-p) 안 쓰면 외부접속이 아예 불가능

👉 **접속:** `http://localhost:8080`

- local host 접속 예시 이미지



---

2. OS 컨테이너 백그라운드 유지

```
docker run -it -d --name ubuntu ubuntu:22.04 bash
```

- **해석:** 우분투를 백그라운드에서 꺼지지 않게 '대기 상태'로 켜둠.
- **핵심:** `-d`를 써서 뒷마당(백그라운드)에서 서버가 계속 돌게 함. `-it` 덕분에 "사용자가 언제든 들어올 수 있게 터미널 준비하고 대기해!"라는 상태가 됨.

👉 뒤에 bash 붙이면 바로 터미널창 열리게 함.

---

3. 실행 중인 컨테이너 접속 (들어가기)

```
docker exec -it ubuntu bash    # execute 실행하다의 약어.
```

- **해석:** 이미 켜져 있는(run 상태) 컨테이너 안으로 직접 들어감. (예시는 ubuntu라는 컨테이너)
- **비유:** `run`은 새 집 짓기, `exec`은 지어진 집 방문하기임.

---

4. 내 파일 공유 (볼륨 마운트)

```
docker run -it --name my_ros -v C:\work\docker_ws:/ros_ws u
buntu:22.04 bash
```

- **해석: [내 컴퓨터 폴더]:[컨테이너 폴더]** 를 하나로 연결.
- **장점:** 컨테이너를 지워도 내 컴퓨터 하드에 작업물이 남음. (ROS 개발 필수)

📋 *docker-compose 요약*

1. `docker-compose.yaml` (설계도)

```yaml
version: "3.8"

services:
  web:
    image: nginx
    ports:
      - "8080:80"
```

- **내용:** 이미지 이름, 포트 번호 등 실행에 필요한 모든 정보를 적어둔 파일임.
- **장점:** 긴 명령어를 매번 칠 필요 없이 파일 하나로 관리함.

👉 docker-compose.yaml 파일은 띄어쓰기에 굉장히 예민함. *(클론, 대시 뒤에는 띄어쓰기로 구분해주기.)*

2. 주요 명령어

```
docker-compose up -d  # (서버 켜기): 설계도대로 컨테이너를 백그라운드에서 실행함.

docker-compose down   # (서버 닫기): 실행 중인 컨테이너를 정지하고 삭제함.
```

📰 *심화 docker-compose.yaml 파일 형식*

```
services:
  ros2:
    image: osrf/ros:humble-desktop  # osrf에서 ros humble과 관련 기본 파일들 가져옴.
    container_name: ros2_container  # 여기에 원하는 이름 작성
    stdin_open: true                # 터미널 입력 열어두기
    tty: true                       # 컨테이너가 안 꺼지게 대기시키기

    volumes:
      - ./workspace:/workspace      # [내 컴퓨터 폴더]:[컨테이너 폴더] 연결

    working_dir: /workspace         # 접속 시 시작할 폴더 지정

    command: bash                   # 시작하자마자 터미널(bash) 실행
```

cf) osrf : Open Source Robotics Foundation : ROS를 만드는 공식 단체 계정

docker-compose.yaml 파일이 있는 곳에서 실행시켜줘야 함.

```
docker-compose up -d
```

📦 *docker-compose 이후, ROS2 패키지 생성 및 빌드 과정 요약*

1. 패키지 생성

```
ros2 pkg create --build-type ament_python my_pkg
```

- **해석:** Python 기반의 ROS2 패키지(`my_pkg`)를 자동으로 만들어줌.
- **결과:** `setup.py`, `package.xml` 등 필수 파일들이 담긴 폴더가 생성됨.

---

2. 빌드 실행

```
colcon build
```

- **명령:** `/workspace` 경로에서 실행함.
- **해석:** 내가 만든 소스코드(`src`)를 컴퓨터가 실행할 수 있는 형태로 변환(컴파일/설치)함.
- **결과:** `build`, `install`, `log` 폴더가 자동으로 생김.
  - `install/`: 실제로 실행될 파일들이 모이는 곳임.
  - `build/`: 빌드 과정 중 발생하는 임시 파일 저장소임.

💡 빌드 후 필수 단계 (중요)

빌드가 끝났다고 바로 실행할 수 있는 게 아님. **내가 만든 패키지를 인식**시켜줘야 함.

```
# /workspace 폴더에서 실행함
source install/setup.bash
```

- **해석:** "방금 빌드해서 `install` 폴더에 새로 생긴 패키지들을 현재 터미널에 등록해라"라는 뜻임.
- **순서:** `colcon build` ➔ `source install/setup.bash` ➔ `ros2 run ...` (이게 한 세트임)

👉 **주의** : src/my_pkg 여기서 노드를 만들어야 하는데, install/my_pkg에서 할 수도 있음.